

Database for Data Engineers

SQLite · Pandas · SQLAlchemy Core · ORM — Sadədən Orta Səviyyəyə

Bu sənəd verilənlər bazasına giriş üçün addım-addım Python bələdçisidir. Hər level öncəkinin üzərində qurulur. SQLAlchemy Core-dan əvvəl sqlite3 + pandas öyrənilir — ORM-i başa düşmək üçün əsas lazımdır.

İçindəkilər

1. Level 1 — sqlite3: Əsas Əməliyyatlar
2. Level 2 — Pandas ilə İntegrasiya
3. Level 3 — SQLAlchemy Core (Real DB bağlantısı)
4. Level 4 — SQLAlchemy ORM (Model, Session)
5. Level 5 — Data Science üçün Əlavələr
- . Quick Reference Card

LEVEL 1 — sqlite3 — Əsas Əməliyyatlar

sqlite3 Python-a daxildir, quraşdırmağa ehtiyac yoxdur. Verilənlər bazasını öyrənmək üçün ən yaxşı başlanğıc nöqtəsidir.

□ **Qeyd:** ':memory:' — RAM-da müvəqqəti DB yaradır (test üçün ideal). Diskə yazmaq üçün 'myfile.db' istifadə edin.

1.1 — Bağlantı və Cursor

```
import sqlite3

# ":memory:" → RAM-da müvəqqəti DB (restart-da silinir)
# "shop.db" → diskdə daimi fayl
connection = sqlite3.connect(":memory:")

# Cursor = sorğu icraçısı (ənənəvi üsul)
cursor = connection.cursor()
```

1.2 — Cədvəl yaratmaq

```
cursor.execute("""
    CREATE TABLE customers (
        id      INTEGER PRIMARY KEY,
        name    TEXT,
        city    TEXT,
        spending REAL
    )
""")
```

□ **Tip:** PRIMARY KEY avtomatik unikal ID verir. REAL = onluq ədəd (float).

1.3 — Məlumat əlavə etmək

```
# Tək sətir
cursor.execute(
    "INSERT INTO customers VALUES (?, ?, ?, ?)",
    (1, "Alice", "New York", 320.50)
)
```

ALT → Çox sətir — executemany (təvsiyə olunur)

```
customers = [
    (1, "Alice", "New York", 320.50),
    (2, "Bob", "London", 89.99),
    (3, "Carol", "New York", 540.00),
    (4, "Dave", "Paris", 15.00),
    (5, "Emma", "London", 210.75),
]
cursor.executemany(
    "INSERT INTO customers VALUES (?, ?, ?, ?)",
    customers
)
connection.commit() # ← Ctrl+S kimidir — unutma!
```

⚠ **Diqqət:** '?' placeholder istifadə et — birbaşa string format etmə! SQL Injection hücumundan qoruyur.

1.4 — Məlumat oxumaq

```
cursor.execute("SELECT * FROM customers")
rows = cursor.fetchall() # Bütün sətirləri al (list of tuples)

for row in rows:
    print(row)
# Nəticə: (1, 'Alice', 'New York', 320.5)
```

ALT → fetchone() — 1 sətir | fetchmany(n) — N sətir

```
# Yalnız 1 sətir (böyük DB-lərdə yaddaşa qənaət)
row = cursor.fetchone()

# Dəqiq N sətir
rows = cursor.fetchmany(3)
```

1.5 — WHERE ilə Filtrləmə

```
# London müştəriləri
cursor.execute(
    "SELECT name, spending FROM customers WHERE city = ?",
    ("London",)
)
for row in cursor.fetchall():
    print(row)
```

ALT → Çox şərt — AND / OR

```
cursor.execute(
    "SELECT * FROM customers WHERE city = ? AND spending > ?",
    ("London", 100)
)
```

1.6 — Bağlantını bağlamaq

```
connection.close()    # Resursu azad et

# — DAHA YAXŞI ÜSUL: context manager —————
with sqlite3.connect("shop.db") as conn:
    cur = conn.cursor()
    cur.execute("SELECT * FROM customers")
    print(cur.fetchall())
# with blokundan çıxanda avtomatik bağlanır
```

□ **Tip:** 'with' bloku həmişə bağlantını bağlayır — istisna baş versə belə. Production kodunda həmişə bu üsuldən istifadə et.

LEVEL 2 — Pandas ilə İnteqrasiya

Ham SQL nəticəsi tuple siyahısıdır — işləmək çətinidir. pandas onu adlı sütunlu DataFrame-ə çevirir.

□ **Qeyd:** pip install pandas sqlalchemy

2.1 — DataFrame → SQL (yazma)

```
import pandas as pd
import sqlite3

connection = sqlite3.connect(":memory:")

df = pd.DataFrame(customers,
                  columns=["id", "name", "city", "spending"])

df.to_sql(
    "customers",          # cədvəl adı
    connection,           # bağlantı
    index=False,          # DataFrame indeksini yazma
    if_exists="replace"   # replace | append | fail
)
```

2.2 — SQL → DataFrame (oxuma)

```
df = pd.read_sql("SELECT * FROM customers", connection)
print(df)
```

#	id	name	city	spending
# 0	1	Alice	New York	320.50
# 1	2	Bob	London	89.99

ALT → Parametrlı oxuma (güvənli üsul)

```
from sqlalchemy import text

df = pd.read_sql(
    text("SELECT * FROM customers WHERE city = :city"),
    connection,
    params={"city": "London"}
)
```

2.3 — pandas əməliyyatları

```
# Sətir filtrləmə
high_spenders = df[df["spending"] > 200]

# Hesablanmış sütun əlavə etmək
df["spending_eur"] = (df["spending"] * 0.92).round(2)

# Şəhərə görə qruplaşdırma + cəm
by_city = df.groupby("city")["spending"].sum().reset_index()
by_city.columns = ["city", "total_spending"]

# Sürətli statistika
print(df["spending"].describe())
# count      5.00
# mean      235.25   std   198.07
# min       15.00   max   540.00
```

2.4 — ETL nümunəsi (Extract → Transform → Load)

ETL data engineering-in əsas dövrüdür: mənbədən oxu, çevir, hədəfə yaz.

```
def run_etl(source_engine, target_engine):

    # 1. EXTRACT — xam məlumatı oxu
    raw = pd.read_sql("SELECT * FROM orders", source_engine)
    print(f"  Extracted {len(raw)} rows")

    # 2. TRANSFORM — təmizlə və zənginləşdir
    df = raw.copy()
    df = df[df["status"] == "shipped"]          # yalnız göndərilənlər
    df["amount_eur"] = (df["amount"] * 0.92).round(2)
    df["customer"] = df["customer"].str.upper() # adları normallaşdır
    print(f"  Transformed → {len(df)} rows")

    # 3. LOAD — hədəf DB-yə yaz
    df.to_sql("shipped_orders", target_engine,
              index=False, if_exists="replace")
    print(f"  Loaded → 'shipped_orders' ✓")

    return df
```

❏ **Tip:** ETL-i real layihədə Apache Airflow, Prefect və ya dbt ilə planlaşdırmaq olar. pandas hissəsi eyni qalır.

LEVEL 3 — SQLAlchemy Core — Real DB Bağlantısı

SQLAlchemy Python-da ən çox istifadə edilən DB kitabxanasıdır. Core hissəsi birbaşa SQL yazmağa imkan verir.

❏ **Qeyd:** `pip install sqlalchemy psycopg2-binary` # PostgreSQL üçün

3.1 — Engine yaratmaq (bağlantı nöqtəsi)

```
from sqlalchemy import create_engine

# SQLite (öyrənmək üçün, fayl əsaslı)
engine = create_engine("sqlite:///shop.db", echo=False)

# SQLite (RAM-da, test üçün)
engine = create_engine("sqlite:///memory:", echo=True)
```

ALT → PostgreSQL / MySQL — real server bağlantısı

```
# PostgreSQL (data engineering-də ən çox istifadə olunan)
engine = create_engine(
    "postgresql+psycopg2://user:password@host:5432/dbname",
    pool_pre_ping=True # bağlantı kəsilmələrini avtomatik həll edir
)

# MySQL
engine = create_engine(
    "mysql+pymysql://user:password@host:3306/dbname"
)
```

⚠ **Diqqət:** Şifrəni kodda yazma! Mühit dəyişəni istifadə et: `import os pw = os.getenv('DB_PASS')` `engine = create_engine(f"postgresql://user:{pw}@host/db")`

3.2 — Bağlantı açmaq və SELECT

```
from sqlalchemy import text

with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM customers"))
    rows = result.fetchall() # list of Row (tuple kimi)
```

ALT → mappings() — dict kimi oxumaq (daha oxunaqlı)

```
with engine.connect() as conn:
    rows = conn.execute(
        text("SELECT * FROM customers")
    ).mappings().all() # [{"id":1,"name":"Alice",...}]

    for r in rows:
        print(r["name"], r["spending"])
```

ALT → Parametrlı sorğu — həmişə bunu istifadə et

```
with engine.connect() as conn:
    result = conn.execute(
        text("SELECT * FROM customers WHERE city = :city AND spending > :min"),
        {"city": "Baku", "min": 200}
    )
    rows = result.fetchall()
```

3.3 — INSERT / UPDATE / DELETE

```
with engine.connect() as conn:
    # INSERT
    conn.execute(text("""
        INSERT INTO customers (name, city, spending)
        VALUES (:name, :city, :spending)
    """), {"name": "Nurlan", "city": "Baku", "spending": 450.75})

    conn.commit()    # Mütləq! Unutsan dəyişikliklər saxlanmaz.
```

ALT → Kütləvi INSERT — siyahı göndər (executemany)

```
new_rows = [
    {"name": "Elchin", "city": "Baku", "spending": 500},
    {"name": "Leyla", "city": "Sumqayıt", "spending": 180},
]
with engine.connect() as conn:
    conn.execute(
        text("INSERT INTO customers (name,city,spending) VALUES (:name,:city,:spending)"),
        new_rows
    )
    conn.commit()
```

```
# UPDATE
with engine.connect() as conn:
    conn.execute(
        text("UPDATE customers SET spending = :s WHERE id = :id"),
        {"s": 650, "id": 5}
    )
    conn.commit()

# DELETE
with engine.connect() as conn:
    conn.execute(
        text("DELETE FROM customers WHERE spending < :min"),
        {"min": 50}
    )
    conn.commit()
```

3.4 — pandas + SQLAlchemy Core


```
# DataFrame → DB
df.to_sql("orders", engine, index=False, if_exists="replace")

# DB → DataFrame
df = pd.read_sql("SELECT * FROM orders", engine)

# Parametrli
df = pd.read_sql(
    text("SELECT * FROM orders WHERE status = :s"),
    engine, params={"s": "shipped"}
)
```

LEVEL 4 — SQLAlchemy ORM — Model və Session

ORM cədvəlləri Python class-larına çevirir. SQL yazmaq əvəzinə Python obyektləri ilə işləyirsən.

4.1 — Model (cədvəl strukturu)

```
from sqlalchemy import Column, Integer, String, Float, ForeignKey
from sqlalchemy.orm import declarative_base, relationship

Base = declarative_base()

class Customer(Base):
    __tablename__ = "customers"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String(100), nullable=False)
    city = Column(String(50), index=True) # axtarış sürəti üçün
    spending = Column(Float, default=0.0)
    email = Column(String(100), unique=True)
    orders = relationship("Order", back_populates="customer")

    def __repr__(self):
        return f""

class Order(Base):
    __tablename__ = "orders"

    id = Column(Integer, primary_key=True)
    customer_id = Column(Integer, ForeignKey("customers.id"))
    amount = Column(Float)
    status = Column(String(20))
    customer = relationship("Customer", back_populates="orders")

# Bütün cədvəlləri DB-də yarat
Base.metadata.create_all(engine)
```

□ **Tip:** 'index=True' — tez-tez axtarılan sütunlara qoy. Böyük cədvəllərdə sorğu sürətini 10x artırır.

4.2 — Session (əməliyyat idarəçisi)

```
from sqlalchemy.orm import sessionmaker

Session = sessionmaker(bind=engine)

# Təvsiyə olunan üsul — context manager
with Session() as session:
    new_customer = Customer(name="Aysel", city="Gəncə", spending=320)
    session.add(new_customer)
    session.commit()
```

ALT → Çox obyekt əlavə etmək — add_all

```
with Session() as session:
    customers = [
        Customer(name="Elçin", city="Bakı", spending=500),
        Customer(name="Leyla", city="Sumqayıt", spending=180),
    ]
    session.add_all(customers)
    session.commit()
```

ALT → Kütləvi insert — bulk_insert_mappings (ən sürətli)

```
with Session() as session:
    session.bulk_insert_mappings(Customer, [
        {"name": "Nurlan", "city": "Bakı", "spending": 450},
        {"name": "Rauf", "city": "Lənkəran", "spending": 290},
    ])
    session.commit()
```

□ **Qeyd:** bulk_insert_mappings event-ləri tetikləmir — validation lazımdırsa add_all istifadə et.

4.3 — Sorğu / Filtrləmə

```
with Session() as session:
    # Bütün müştərilər
    all_c = session.query(Customer).all()

    # Bir şərtlə
    baku = session.query(Customer).filter(Customer.city == "Bakı").all()

    # Çox şərt
    top_baku = session.query(Customer).filter(Customer.spending > 300,
                                                Customer.city == "Bakı").all()

    # Sıralama + limit
    top5 = session.query(Customer).order_by(Customer.spending.desc()).limit(5).all()

    # İlk nəticə
    first = session.query(Customer).filter_by(name="Aysel").first()
```

ALT → SQLAlchemy 2.0+ stili — select() (müasir)

```
from sqlalchemy import select

with Session() as session:
    stmt = select(Customer).where(Customer.city == "Bakı")
    baku = session.execute(stmt).scalars().all()

    stmt = select(Customer).order_by(Customer.spending.desc()).limit(10)
    top10 = session.execute(stmt).scalars().all()
```

4.4 — UPDATE və DELETE

```
with Session() as session:
    # UPDATE – obyektı tap, dəyişdir, commit et
    c = session.query(Customer).filter_by(id=5).first()
    if c:
        c.spending = 650
        c.city = "Bakı"
        session.commit()

    # DELETE
    session.query(Customer).filter(Customer.spending < 50).delete()
    session.commit()
```

4.5 — JOIN və GROUP BY

```
from sqlalchemy import func, select

with Session() as session:
    # JOIN – müştəri adı + sifariş məbləği
    result = session.execute(
        select(Customer.name, Order.amount)
        .join(Order)
        .where(Customer.city == "Bakı")
    )
    for name, amount in result:
        print(name, amount)

    # GROUP BY + aggregate
    stmt = select(
        Customer.city,
        func.sum(Customer.spending).label("total_spend"),
        func.count(Customer.id).label("customer_count")
    ).group_by(Customer.city)

    for city, total, count in session.execute(stmt):
        print(f"{city}: {total:.2f} AZN ({count} müştəri)")
```

LEVEL 5 — Data Science üçün Əlavələr

Data Scientist-lərin gündəlik işdə ən çox ehtiyac duyduğu verilənlər bazası əməliyyatları.

5.1 — Böyük cədvəlləri chunks ilə oxumaq

Milyonlarla sətirlik cədvəl birdən RAM-a yükləmək çökməyə səbəb ola bilər. chunksize parametri məlumatı hissə-hissə oxuyur:

```
# 50 000 sətirlik hissələrlə oxu
for chunk in pd.read_sql("SELECT * FROM big_table", engine, chunksize=50_000):
    # Hər chunk ayrıca DataFrame-dir
    processed = chunk[chunk["value"] > 0]
    processed.to_sql("result", engine, if_exists="append", index=False)

print("Bitdi ✓")
```

□ **Tip:** chunksize + if_exists='append' kombinasiyası böyük ETL pipeline-ların əsas sxemidir.

5.2 — SQL ilə Feature Engineering

Ağır hesablamaları pandas-a yükləmək əvəzinə birbaşa SQL-də et — çox daha sürətlidir:

```
query = """
    SELECT
        customer_id,
        COUNT(*)                AS order_count,
        SUM(amount)              AS total_spent,
        AVG(amount)              AS avg_order,
        MAX(amount)              AS max_order,
        MIN(order_date)          AS first_order,
        MAX(order_date)          AS last_order,
        julianday('now') - julianday(MAX(order_date)) AS days_since_last_order
    FROM orders
    WHERE status = 'shipped'
    GROUP BY customer_id
"""
features_df = pd.read_sql(query, engine)
```

□ **Qeyd:** Bu pattern RFM (Recency, Frequency, Monetary) analizinin əsasıdır — müştəri segmentasiyasında çox istifadə edilir.

5.3 — Train/Test bölüb DB-də saxlamaq

```
from sklearn.model_selection import train_test_split

df = pd.read_sql("SELECT * FROM features", engine)
train, test = train_test_split(df, test_size=0.2, random_state=42)

# Hər hissəni DB-də ayrı cədvəl kimi saxla
train.to_sql("features_train", engine, index=False, if_exists="replace")
test.to_sql("features_test", engine, index=False, if_exists="replace")
print(f"Train: {len(train)} | Test: {len(test)}")
```

5.4 — CSV / Excel / Parquet ixrac

```
df = pd.read_sql("SELECT * FROM results", engine)

df.to_csv("output.csv", index=False)           # universal, yüngül
df.to_excel("output.xlsx", index=False,        # biznes hesabatları
            sheet_name="Results")
df.to_parquet("output.parquet", index=False)    # pip install pyarrow
```

□ **Tip:** Parquet = sütun əsaslı format. pandas, Spark, DuckDB, BigQuery hamısı dəstəkləyir. CSV-dən 5-10x daha sürətli oxunur.

5.5 — DuckDB ilə sürətli analitik sorğular

DuckDB — DataFrame-lər üzərində birbaşa SQL icra edir, xarici server tələb etmir:

```
# pip install duckdb
import duckdb

df = pd.read_sql("SELECT * FROM orders", engine)

# DataFrame-i SQL ilə birbaşa sorğulamaq!
result = duckdb.query("""
    SELECT customer, SUM(amount) AS total
    FROM df
    GROUP BY customer
    ORDER BY total DESC
""").df()           # .df() → pandas DataFrame qaytarır

print(result)
```

□ **Qeyd:** DuckDB milyonlarla sətiri saniyələr içində analiz edir və çox iş parçacığını dəstəkləyir.

Quick Reference Card

Bağlan (SQLite)	<code>sqlite3.connect('file.db')</code>
Bağlan (istənilən DB)	<code>create_engine('dialect://...')</code>
Cədvəl yarat (ORM)	<code>Base.metadata.create_all(engine)</code>
DataFrame → DB	<code>df.to_sql('table', engine, index=False)</code>
DB → DataFrame	<code>pd.read_sql('SELECT ...', engine)</code>
Sətir filtrləmə	<code>df[df['col'] > value]</code>
Sütun əlavə et	<code>df['new'] = df['old'] * 2</code>
Qruplaşdır + cəm	<code>df.groupby('col')['val'].sum()</code>
Sürətli statistika	<code>df.describe()</code>
CSV ixrac	<code>df.to_csv('out.csv', index=False)</code>
Parquet ixrac	<code>df.to_parquet('out.parquet')</code>
Chunks ilə oxu	<code>pd.read_sql(..., chunksize=50_000)</code>
ORM — əlavə et	<code>session.add(obj); session.commit()</code>
ORM — filtrə	<code>session.query(Model).filter(...).all()</code>
ORM — sil	<code>session.query(Model).filter(...).delete()</code>

Növbəti addımlar

- **pandas sənədləri** — <https://pandas.pydata.org/docs/>
- **SQL məşq saytı** — <https://sqlzoo.net>
- **SQLAlchemy sənədləri** — <https://docs.sqlalchemy.org>
- **DuckDB** — <https://duckdb.org>
- **Pulsuz PostgreSQL** — <https://neon.tech>
- **Airflow (ETL planlaması)** — <https://airflow.apache.org>